Department of Artificial Intelligence and Machine Learning

Acharya Institute of Technology

Soladevanahalli, Bengaluru 560 107

Project Phase-II · Interim Presentation

on [18/08/20226]

TraceWeave-Core: Measuring What Static Call-Graph Extraction Misses in Real Python Repositories

Presented by

Group No	Name	USN
28	Abhishek Kumar	1AY23AI004
	Pravith S Upadhyaya	1AY23AI086
	Sankalp Shankar	1AY23AI101
	Surabhi Singh	1AY23AI125
Guide Name: **Designation:**		

ACHARYA

Dept of AI &ML

TABLE OF CONTENTS

No.	Contents	Page
1	Problem Statement & Objective	3
2	Flow / Methodology	4–5
3	Demo of Project	6
4	Results / Output	7
5	Status of Paper	8–9
6	Proposed Outcome vs Actual Outcome	10–11
7	Conclusion, Deliverables & Impact	12
8	Individual Responsibilities	13
9	Gantt Chart	14
10	References	15

ACHARYA

Dept of AI & ML

1. Problem Statement & Objective

ACHARYA

Dept of AI & ML

THE PROBLEM

- Repository-level modernization is bottlenecked by call-graph quality, not by model capacity.
- Python decides many calls at run time — getattr dispatch, importlib, plugin registries, DI containers.
- Static extractors cannot see those edges, and no one has measured how many are lost.
- A silently lossy call graph makes an LLM code agent confidently wrong.

WHAT ONE MISS LOOKS LIKE

- Static extraction (PyCG 0.0.8) reports two edges: main → dispatch, then dispatch → <builtin>.getattr.
- Runtime tracing (sys.setprofile) reports two edges: main → dispatch, then dispatch → handle_email.
- The second runtime edge is real, is executed on every run, and is invisible to static analysis.

OBJECTIVES

- O1 — Measure the miss rate: recall of static extraction against runtime-observed edges (hypothesis H2a).
- O2 — Name the mechanism: show that misses concentrate in identifiable construct classes, not diffusely.
- O3 — Release DynBench-Core: pinned repositories, replayable workloads, reference edge sets and harness.
- O4 — Report feasibility: what it actually costs to trace real projects at corpus scale.

2. Flow / Methodology — Measurement Pipeline (1 of 2)

ACHARYA

Dept of AI & ML

THE PIPELINE — SIX STAGES, EACH WITH A FIXED OUTPUT

- Discover — GitHub search over Python repositories that carry a build file. Output: 558 candidates queued.
- Score density — AST scorer counts dynamic constructs per KLOC of source only. Output: LOW / MED / HIGH stratum.
- Triage — containerised install and test run; one repo, one Python, one attempt. Output: GREEN / AMBER / RED plus a pinned lockfile.
- Capture workloads — annotators record at least three replayable runs per repo, verified twice. Output: the execution oracle.
- Extract both graphs — static: PyCG 0.0.8 and Pyright 1.1.411; dynamic: the sys.monitoring tracer. Output: S and R.
- Fuse and measure — dual-state CPG fused on read; recall computed over the comparable universe. Output: the H2a number.

WHY THE ORDER IS PART OF THE METHOD

- Density scoring precedes triage, so the corpus is stratified before expensive container work is spent on it.
- Workload capture precedes extraction, so the runtime edge set is fixed by a recorded oracle rather than by whatever happened to execute.
- Fusion is last, so neither graph can be adjusted after the other has been seen.

EXECUTION ENVIRONMENT

- Every stage runs inside a container pinned by digest, never by tag.
- The host never traces — it only schedules, collects and verifies.

2. Flow / Methodology — What Makes It a Measurement (2 of 2)

ACHARYA

Dept of AI & ML

IDENTITY IS PINNED BEFORE ANYTHING RUNS

- Repo identity requires all five fields: repo_url, sha, pyver, base_image_digest, lockfile_sha256.
- Node identity is frozen as: repo_url, sha, relative_path, qualified_symbol.
- Line numbers are deliberately excluded — they move under reformatting and differ between the static parse and the runtime frame.
- Both graphs are extracted inside the same pinned image, so a difference between them is a property of the code, not of the environment.

METRIC DISCIPLINE, PRE-REGISTERED

- Comparable universe — both endpoints in-repo and Python-level; C/builtin and third-party edges are excluded from the primary denominator.
- Recall = $|S \cap R| / |R|$. The quantity $|S \cap R| / |S|$ is never called precision; it is the workload-confirmed fraction.
- R is a lower bound, conditional on the recorded workloads. That wording is mandatory everywhere it is reported.
- Wrapper transparency — caller → wrapper → f collapses to caller → f, so decorators cannot manufacture misses.
- No LLM-as-judge anywhere in validation or in any headline metric.

FROZEN BEFORE THE CODE THAT CONSUMES THEM

- metric_spec_h2a.md, frozen v1.0 — node identity, what counts as an edge, the comparable universe, threats pre-empted.
- density_scorer_spec.md, frozen v1.0 — thresholds calibrated once, on 14 reference repos, before any corpus admission.
- config/base_images.json, frozen v1.2 — five image digests, tool pins, the recipe ladder; appended to, never edited.

3. Demo of Project — A Call Graph Missing a Real Call

ACHARYA

Dept of AI & ML

WHAT RUNS ON SCREEN

- fixture.py — a small plugin registry that dispatches through getattr, the exact construct class H2a is about.
- PyCG 0.0.8 — runs in the pinned container against the same file and writes static.json.
- sys.setprofile tracer — executes the fixture for real and writes runtime.json.
- Web explainer — React and Cytoscape diff the two graphs and highlight the edge only one of them has.

REAL OUTPUT FROM THIS REPOSITORY

- static.json, 3 edges: fixture → fixture.main; fixture.main → PluginRegistry.dispatch; PluginRegistry.dispatch → <builtin>.getattr.
- runtime.json, 3 edges: fixture → fixture.main; fixture.main → PluginRegistry.dispatch; PluginRegistry.dispatch → PluginRegistry.handle_email.
- The two graphs agree on the first two edges and disagree on the third.
- PluginRegistry.handle_email is executed on every run and appears in no static graph.

DEMO SAFETY

- A warm end-to-end run takes 0.35 to 2.7 seconds; the fallback threshold is 15 seconds.
- restore_fallback.sh replays a recorded static/runtime pair if the live run does not complete.

WHY THE DEMO IS THE ARGUMENT

- The demo is the hypothesis in miniature: one file, one construct, one lost edge.
- Run the same static-versus-runtime diff across the whole pinned corpus and the result is the H2a number.

4. Results / Output — Phase 0, Measured

ACHARYA

Dept of AI & ML

HEADLINE NUMBERS

- 558 candidates discovered and scored, against a target of ~450 and a floor of 300.
- 100% PyCG failure rate under Python 3.12 — it does not run there at all.
- 1.6% PyCG failure on library code under Python 3.10, and 3.1% on test and fixture paths.
- 93 seconds to triage click GREEN, end to end, producing a 9-package lockfile and its sha256.

STRATIFIED QUEUE AND SCORER CALIBRATION

- HIGH, density ≥ 10.0 — 135 repos queued, 6 in the reference set.
- MED, density 4.0 to 10.0 — 177 repos queued, 4 in the reference set.
- LOW, density < 4.0 — 246 repos queued, 3 in the reference set.
- TOO_SMALL, under 500 SLOC — not queued, 1 in the reference set.
- Totals: 558 repos queued; 14 reference repos used for calibration.
- Stratum is computed from source only — scoring test suites together made every pytest project look HIGH.

FOUND BY RUNNING PHASE 0, NOT BY PLANNING IT

- PyCG 0.0.8 cannot be imported on Linux — it ships PyCG/ but imports pycg; a symlink shim is now mandatory in every image.
- The scorer's first run had no discrimination: 10 of 14 repos landed HIGH. Four false-positive classes were fixed before freezing.
- The scorer corrected two a-priori labels and was right — requests really does call importlib.import_module in its compat layer.
- ins1's classifier has an ordering bug. It is kept verbatim as error_class_v10 for audit comparability, and the disagreement rate is now reportable.
- A 16.8× overcount was caught in verification — click's collected tests read as 32965 instead of 1965.

5. Status of Paper (1 of 2)

ACHARYA

Dept of AI & ML

PAPER 1 · SYSTEMATIC REVIEW · UNDER REVIEW

- Title: “Graph-Aware Neuro-Symbolic Approaches for Legacy Modernization: A Systematic Review”.
- Authors: Adarsha S P · Sankalp Shankar · Surabhi Singh · Pravith S Upadhyaya · Abhishek Kumar.
- Sources searched: IEEE Xplore, ACM Digital Library and arXiv, January 2020 to April 2026.
- Subject: repository-level code translation, refactoring and validation.
- §VIII.B is the finding this project turns into a measurement.
- Venue: [Venue Name]. Submitted: [Date]. Paper ID: [ID].

PAPER 2 · IN PREPARATION

- Title: “TraceWeave-Core: How Much of a Python Call Graph Does Static Extraction Miss?”
- Gated on the Phase 3 GO / NO-GO decision.
- Headline result: the H2a recall figure, with its per-stratum and per-edge-type breakdowns.
- Accompanying artifact: DynBench-Core.
- The measurement is publishable either way — a low miss rate concentrated at named constructs is a stronger result than a diffuse high one.

PRE-REGISTRATION

- No threshold or definition may be tuned after seeing results — the specs were frozen before the extraction code existed.

5. Status of Paper — Proof (2 of 2)

ACHARYA

Dept of AI & ML

SUBMISSION PROOF

- The submission-confirmation screenshot is pasted onto this slide before the review.
- Acceptable proof: the acknowledgement e-mail, or the portal's submitted-papers page.

WHAT THE PROOF MUST SHOW

- Paper title, exactly as submitted.
- Full author list, in submission order.
- Venue name and track.
- Submission or paper ID.
- Date-stamped acknowledgement.
- Current state, as reported by the portal.

BEFORE THE SLIDE IS SHOWN

- Redact reviewer identities.
- Redact any e-mail address that is not a team member's.
- Confirm the screenshot is legible when projected, not only on a laptop screen.

6. Proposed Outcome vs Actual Outcome (1 of 2)

ACHARYA

Dept of AI & ML

PHASE 0 COMMITMENTS — MET

- Candidates discovered and scored — proposed ~450, floor 300; actual 558 queued.
- Stratum balance, HIGH not thin — risk flagged in the plan; actual HIGH 135 / MED 177 / LOW 246.
- Pyright pinned before Phase 1 — version query pending; actual 1.1.411, published 2026-06-25.
- PyCG parse-failure rate measured — proposed as “expected non-trivial”; actual 1.6% at 3.10 and 100% at 3.12.
- triage/runner.py parallel queue — proposed recipe ladder plus checkpoints; delivered, and it re-verifies every outcome.
- Density scorer calibrated and frozen — proposed calibrate once then freeze; delivered v1.0, 6 HIGH / 4 MED / 3 LOW / 1 TOO_SMALL.

COMMITMENTS CARRIED FORWARD — NEXT

- Corpus with at least three workloads per repo — proposed ~60 repos, floor 40; Phase 1 work, not started.
- H2a recall figure — proposed at the Phase 3 GO / NO-GO; not yet measured, by design.

HOW TO READ THIS

- Phase 0 closed on schedule and over target on every item it owned.
- The two NEXT items are Phase 1 and Phase 3 deliverables, not slippage.

6. What the Plan Did Not Predict (2 of 2)

ACHARYA

Dept of AI & ML

FOUR SURPRISES, AND WHAT EACH ONE CHANGED

- PyCG does not run on Python 3.12 at all — 1311 of 1311 reference files fail with ImportManagerError, not a SyntaxError, so §2 anticipated the wrong risk.
  - Response: static baselines now run on Python ≤ 3.11 while repos are triaged on 3.12, recorded as a deliberate split rather than discovered from a stack trace.
- The published 0.0.8 sdist is unimportable on Linux — it installs PyCG/ but every internal import reads pycg, so it only ever worked on case-insensitive macOS.
  - Response: the pin is held at 0.0.8, every image carries the symlink shim, and the deviation is written into MANIFEST §2.1.
- This Mac cannot build the corpus — 8 GB of RAM OOM-kills test suites at --jobs 3, and the corpus must be built on one architecture.
  - Response: the Mac is the manager host and the Linux x86_64 box executes; the corpus is amd64, and a Mac-triaged repo counts as development output.
- A patch introduced a silent 16.8× overcount — n_collected matched pytest's pre-deselection total, so click read 32965 instead of 1965.
  - Response: caught during verification and fixed in both triage.sh and runner.py, with the pre-deselection total kept separately as n_collected_total.

STILL OPEN — CARRIED INTO PHASE 1

- Amend ins1 §6: triage.sh v1.2 emits OOM, TIMEOUT, KILLED and collect_partial, which the operator sheet forbids — an operator cannot record what the script prints.
  - validate_sheet.py --v12 is the interim machine check until the amendment lands.
- Re-run the 10% human-versus-classifier audit under v1.2.
- Do not pool the 14 v1.1 spot-check rows with the 558.

7. Conclusion, Deliverables & Impact

ACHARYA

Dept of AI & ML

KEY FINDINGS

- Phase 0 closed complete and over target: 558 candidates against a plan of ~450.
- The field's standard static baseline fails on 100% of files under the interpreter this project defaults to.
- Dynamic-feature density is measurable and discriminating once four false-positive classes are removed.
- Every headline defect so far was found by running the pipeline, not by reviewing it.

DELIVERABLES

- DynBench-Core — pinned repositories, replayable workloads, reference edge sets and harness.
- A pre-registered H2a metric specification, frozen before any extraction code existed.
- Automated triage — triage.sh v1.2 and runner.py, with explicit corpus admission.
- A tracing-feasibility survey, including the repositories that could not be traced.

IMPACT

- Tells graph-conditioned code agents how lossy their retrieval substrate really is.
- Names the construct classes to fix, rather than reporting one diffuse percentage.
- Benchmarks outlive methods — the artifact stays useful after the number is quoted.
- A negative result is still the paper: the conditional claim is the strong one.

WHY THE PINS EXIST

- Every pin, digest and frozen threshold exists so the published number is a measurement, not the result of searching until something looked good.

8. Individual Responsibilities

ACHARYA

Dept of AI & ML

WORKSTREAM OWNERSHIP

- [Member 1 Name], [1AY__AI___] — Discovery & stratification. Owns discover_candidates.py, density_score.py, scorer calibration and the frozen thresholds.
- [Member 2 Name], [1AY__AI___] — Triage automation. Owns triage.sh v1.2, runner.py, lockfile pinning, corpus admission and the 10% audit.
- [Member 3 Name], [1AY__AI___] — Static baselines. Owns PyCG 0.0.8 and Pyright 1.1.411 in pinned images, parse-rate measurement and the feasibility survey.
- [Member 4 Name], [1AY__AI___] — Tracing, demo & specification. Owns the sys.monitoring tracer, the live demo and web explainer, metric_spec_h2a.md and MANIFEST.

GUIDE — [GUIDE NAME], [DESIGNATION]

- Reviews every freeze and every version bump.
- Approves the Phase 3 GO / NO-GO.
- Supervises submission.

SHARED ACROSS ALL FOUR MEMBERS

- Weekly integration.
- The 10% human-versus-classifier audit.
- Rehearsal of the live demo on the presentation machine.

9. Gantt Chart — Workstreams



Eight of twelve workstreams are delivered, in the first three weeks. All four of MANIFEST §7's Phase-1 blockers — the Pyright pin, the PyCG parse-rate measurement, runner.py and discover_candidates.py — were cleared before Phase 1 opened.

The chart stops at the Week 8 seam because H2a, DynBench-Core and the feasibility survey already form a complete paper there. H2b-lite and the release run weeks 8–12 beyond it.

ACHARYA

Dept of AI & ML

10. References

ACHARYA

Dept of AI & ML

IEEE FORMAT

[1] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, “PyCG: Practical call graph generation in Python,” in Proc. 43rd Int. Conf. Software Engineering (ICSE), 2021, pp. 1646–1657.

[2] A. S. P, S. Shankar, S. Singh, P. S. Upadhyaya, and A. Kumar, “Graph-aware neuro-symbolic approaches for legacy modernization: A systematic review,” 2026, under review.

[3] M. Shannon, “PEP 669 — Low impact monitoring for CPython,” Python Enhancement Proposals, Python Software Foundation, 2022.

[4] R. Bairi, A. Sonwane, A. Kanade, S. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, “CodePlan: Repository-level coding using LLMs and planning,” in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[5] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, “RepoGraph: Enhancing AI software engineering with repository-level code graph,” in Proc. Int. Conf. Learning Representations (ICLR), 2025.

[6] Y. Pan, J. Zhao, C. Wang, and Q. Liu, “RepoTransBench: A real-world benchmark for repository-level code translation,” arXiv:2412.17744, 2024.

[7] A. R. Ibrahimzada, K. Ke, M. Pawagi, M. S. Abid, R. Pan, S. Sinha, and R. Jabbarvand, “Repository-level compositional code translation and validation,” arXiv:2410.24117, 2024.

[8] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, “GraphCodeBERT: Pre-training code representations with data flow,” in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[9] Microsoft, “Pyright: Static type checker for Python,” version 1.1.411, 2026. [Online]. Available: https://github.com/microsoft/pyright

[10] Python Software Foundation, “ast — Abstract syntax trees,” Python 3.12 documentation, 2026. [Online]. Available: https://docs.python.org/3.12/library/ast.html

NOTE

- Internal project documents (MANIFEST.md, metric_spec_h2a.md, density_scorer_spec.md) are cited in the report, not here.

ACHARYA

Dept of AI & ML

Thank you